

AE 3330

Simulated Orbital Mechanics

By: Geoffrey Hendricks

Section A – John Dec

Fall Semester 2021

In this report, the application of orbital mechanics through a simulated scenario via MATLAB depicts how proper implementation of orbital elements can be used to track and predict the motion of spacecrafts provided few key elements such as time, range, and range rate. The provided scenario tracks a satellite from the NORAD tracking site at Peterson Air Force Base near Colorado Springs that appears to be in a Molniya Orbit. This scenario is modeled through Keplerian equations and adjusted to a Planet Centered Coordinate frame (Earth) through various matrix rotations. The equations used were modeled in MATLAB script using the canonical unit system. This script was used to calculate the initial position and velocity vectors of the satellite, the orbital elements of the satellite at the time of the observation, the true anomaly of the satellite two hours later through iteratively solving Kepler's problem, and the final position and velocity vectors at the new true anomaly. The main premise of this paper is to discuss the importance of being able to account for a spacecrafts trajectory quickly and accurately using the knowledge of orbital mechanics discussed in AE3330.

Nomenclature

L = geodetic latitude

θ = sidereal time

λ_E = geographic longitude relative to Greenwich, measured westward

r_E = radius of the Earth

ρ = range

r = position

v = velocity

$\omega_{\oplus}$ = angular velocity of the Earth

μ = standard gravitational parameter

h = specific angular momentum

e = eccentricity

p = semi-latus rectum

a = semi-major axis

i = inclination

Ω = longitude of ascending node

ω = argument of periapsis

v = true anomaly

n = mean motion

P = orbital period

k = number of periapsis passages

E = eccentric anomaly

M = mean anomaly

I. Introduction

The study of orbital mechanics and the equations derived from it such as Kepler's equations are commonly used by experts at NASA as well as many other space affiliated organizations like SpaceX and Blue Origin. Such knowledge is fundamental for all space travel and with expensive equipment and even lives on the line, it's important to be able track and predict a spacecraft's trajectory in an accurate and timely manner should any potential problems arise. Much of orbital mechanics is built on century old ideas proposed by individuals such as Tycho Brahe, Johannes Kepler, and Isaac Newton. Kepler proposed the most accurate model to fit the observed motion of the planets and developed what is now known as Kepler's equations that are used to determine the final position and velocity of the satellite in the given simulated scenario. To formulate an equation to best fit the observed motion of the planets, Kepler assumed 2-body system in which the force of gravity was the only external force acting on an orbiting body. Because of assumptions like this, there is a slight amount of error that increase over the course of the time being observed. However, this model still holds to be fairly accurate, and error can now be reduced due to current technology like live tracking from tracking stations that can update the testing window between calculations eliminating any major errors.

II. Approach

A. Determining Initial Position and Velocity

To determine the initial position and velocity vectors of the satellite, it was necessary to compute the Julian Day given the time of the year, month, and day of the specific scenario which in this case was September 22nd, 2021. Since the equation for calculating the Julian Day requires just the integer value of

some of the components, we implemented the use of the floor function in MATLAB to round down to a singular whole number value to provide the most accurate Julian Day possible. After the Julian Day was calculated, we could calculate the Julian centuries which was necessary to compute θ_{go} . To calculate θ_g we converted the given time of 5:17pm MST to Universal Time (UT) by adding 7 hours and converted our answer solely to seconds. Once that was done, θ_g could be calculated using the time in UT and θ_{go} . With our λ_E of 104.695°W (longitudinal distance of the NORAD tracking site) provided in the simulated scenario we could find a value for θ using Equation 1 below.

$$\theta = \theta_g + \lambda_E \quad (1)$$

Now with a calculated value for θ and a given latitude value of 38.8326°N it was possible to create a conversion matrix, D, with which the provided range and range rate of the satellite in the Xh,Yh,Zh coordinate frame could be transferred to the planet centric X,Y,Z coordinate from. The equation used to calculate the matrix D is provided below for reference (Equation 2).

$$[D] = \begin{bmatrix} \sin L \cos \theta & \sin L \sin \theta & -\cos L \\ -\sin \theta & \cos \theta & 0 \\ \cos L \cos \theta & \cos L \sin \theta & \sin L \end{bmatrix} \quad (2)$$

It's important to note that all calculations were done in degrees and as such, the function cosd and sind were implemented in MATLAB to assure the correct value of D when applying the matrix conversion. The range and range rate were transferred to the X,Y,Z coordinate frame using the transpose of D as shown in Equation 3 below.

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = [D]^T \begin{bmatrix} Xh \\ Yh \\ Zh \end{bmatrix} \quad (3)$$

With the new X,Y,Z coordinate frame range and range rate the values of the initial position and velocity vectors could finally be calculated using Equations 4-6.

$$\vec{r} = \overline{R}\vec{e} + \vec{\rho} \quad (4)$$

$$\overline{R}\vec{e} = r_e \hat{z} \quad (5)$$

$$\vec{v} = \dot{\vec{\rho}} + \omega_{\oplus} \times \|\vec{r}\| \quad (6)$$

B. Determining Orbital Elements

With the position and velocity vectors calculated in the previous section, the first step to determining the orbital elements was to obtain the values of the three fundamental vector, $\vec{h}$, $\vec{e}$, and $\vec{n}$. Each of the three fundamental vectors could be calculated with the provided position and velocity vectors or vector magnitudes as well as the gravitational parameter of Earth taken to be 1 in the canonical unit system. The equations used are provided below (Equation 7-9).

$$\vec{h} = \vec{r} \times \vec{v} \quad (7)$$

$$\vec{n} = \hat{z} \times \vec{h} \quad (8)$$

$$\vec{e} = \frac{1}{\mu}(\vec{v} \times \vec{r} \times \vec{v}) - \frac{\vec{r}}{\|\vec{r}\|} \quad (9)$$

Once all three fundamental vectors are found, the rest of the necessary orbital elements can be calculated using their component and magnitude form. The final orbital elements were computed using Equations 10-17. Note that any values with subscripts x, y, or z refer to the corresponding component value of the vector referenced.

$$e = \|\vec{e}\| \quad (10)$$

$$a = \frac{h^2}{\mu(1-e^2)} \quad (11)$$

$$p = a * (1 - e^2) \quad (12)$$

$$i = \cos^{-1}\left(\frac{h_z}{h}\right) \quad (13)$$

$$\Omega = \cos^{-1}\left(\frac{n_x}{n}\right) \quad (14)$$

$$\omega = \cos^{-1}\left(\frac{\vec{n} \cdot \vec{e}}{ne}\right) \quad (15)$$

$$\nu = \cos^{-1}\left(\frac{\vec{e} \cdot \vec{r}}{er}\right) \quad (16)$$

$$u = \cos^{-1}\left(\frac{\vec{n} \cdot \vec{r}}{nr}\right) \quad (17)$$

Since it was necessary to adjust the plane in which both ω and ν of the satellite where in, both values were subtracted from 360° to give the correct argument of periapsis and true anomaly.

C. Determining True Anomaly 2hrs Later

In order to obtain the true anomaly two hours after loss of site of the satellite, an iterative approach to Kepler's problem was implemented through MATLAB code. The goal was to start with an initial mean anomaly (Mo) that could be calculated using the mean motion and time of flight of the satellite (TOF). These values could be found using the following three equations (Equations 18-20).

$$P = 2\pi \sqrt{\frac{a^3}{\mu}} \quad (18)$$

$$n = \sqrt{\frac{\mu}{a^3}} \quad (19)$$

$$Mo = n * TOF \quad (20)$$

The next step was defining a trial value to iterate with denoted "Ei" in our MATLAB script which was given a value of 0.5rad. Once the trial value was chosen, a for loop was created in the MATLAB script to iteratively solve for and adjust the value of the eccentric anomaly until the value of E converged for the given time of flight from the initial value of ν :

$$M_i = E_i - e \sin(E_i), \quad dM = M - M_i, \quad \frac{dM}{dE} = 1 - e \cos(E_i), \quad E_{i+1} = E_i + dM \left(\frac{dM}{dE} \right)^{-1} \quad (21-24)$$

Note that all trig functions in the above equations deal with E in radians and not degrees. Once a converged value of E was calculated, the last step was to calculate the new true anomaly using Equation 25 and convert the output from radians to degrees.

$$\nu = \cos^{-1} \left(\frac{\cos(E) - e}{1 - e \cos(E)} \right) \quad (25)$$

D. Determining Final Position and Velocity.

With the new true anomaly determined, it was now possible to predict the final position and velocity vectors of the satellite two hours after loss of sight. To do this, it was necessary to start by calculating the

position and velocity vectors in the perifocal coordinate system using the new true anomaly. To begin, the magnitude of the position vector from the central body to the orbiting body can be calculated with the new ν using Equation 26.

$$r = \frac{P}{1 + e \cos \nu} \quad (26)$$

With this, the position and velocity vectors could be defined in the perifocal coordinate frame shown below (Equations 27 and 28).

$$\vec{r}_w = \langle r \cos \nu, r \sin \nu, 0 \rangle \quad (27)$$

$$\vec{v}_w = \sqrt{\frac{\mu}{p}} \langle -\sin \nu, e + \cos \nu, 0 \rangle \quad (28)$$

After obtaining the position and velocity vectors in the perifocal coordinate system, the next step was to relate the perifocal system to the Earth-centered, equatorial coordinate system as requested in the problem statement. To do this, we defined a conversion matrix, R , that would allow us to multiply our vectors in the perifocal coordinate frame by R to transfer to the equivalent Earth-centered, equatorial coordinate frame. The matrix R used for conversion is defined below (Equation 29).

$$[R] = \begin{bmatrix} \cos \Omega \cos \omega - \sin \Omega \sin \omega \cos i & -\cos \Omega \sin \omega - \sin \Omega \cos \omega \cos i & \sin \Omega \sin i \\ \sin \Omega \cos \omega + \cos \Omega \sin \omega \cos i & -\sin \Omega \sin \omega + \cos \Omega \cos \omega \cos i & -\cos \Omega \sin i \\ \sin \omega \sin i & \cos \omega \sin i & \cos i \end{bmatrix} \quad (29)$$

Multiplying the transformation matrix by $\vec{r}_w$ and $\vec{v}_w$ results in finding $\vec{r}$ and $\vec{v}$ in the geocentric frame at the new value of ν .

III. Results and Discussion

Using MATLAB, a script was written following the described approach above that implemented the given parameters from the simulated scenario as inputs to achieve the desired outputs as requested. The results from each section are listed below.

Initial Position and Velocity Vectors in the Earth-Centered, Equatorial Coordinate System

- $\vec{r} = \langle -2.513872, -0.519053, 0.517741 \rangle$ DU
- $\vec{v} = \langle 0.442306, -0.155707, -0.551056 \rangle$ DU/TU

Orbital Elements of Satellite at Time of Observation

- $e = 0.744231$
- $a = 4.162717$ DU
- $p = 1.8571$ DU
- $i = 62.889800$ deg
- $\Omega = 17.593144$ deg
- $\omega = 280.167920$ deg
- $\nu = 246.998422$ deg

Final True Anomaly 2hrs After Loss of Sight

- $\nu = 129.4426$

iteration	E_i	dM	$\frac{dM}{dE}$	E_{i+1}
0	0.5	0.9075	0.3469	3.1163
1	3.1163	-2.0468	1.7440	1.9427
2	1.9427	0.1986	1.2705	1.7864
3	1.7864	-0.0086	1.1592	1.7789
4	1.7789	2.0147e-05	1.1538	1.7789
5	1.7789	-1.1101e-10	1.1538	1.7789

Predicted Final Position and Velocity Vectors

- $\vec{r} = \langle 1.8063, 1.8554, 2.3882 \rangle$ DU
- $\vec{v} = \langle -0.0991, 0.2420, 0.5091 \rangle$ DU/TU

In the scenario description, the satellite was said to appear moving along a Molniya orbit. Using the results from the orbital elements section, we could test the validity of this statement. A Molniya orbit can be described as having a highly elliptical orbit with an inclination of about 63.4 degrees. When compared to the inclination of the simulated orbit, the percent difference is less than 1. Additionally, Molniya orbits have an argument of periapsis of about 270 degrees and an orbital period close to half a sidereal day which

is about 53.398 TU in canonical units. With an argument of periapsis of 280.168 for the simulated case as well as a period of 53.36 TU, it is within reason to assume the orbit most resembles a Molniya orbit.

While using MATLAB to compute the desired results, there is a fair possibility for slight error in calculated values versus the expected values for the simulation due to certain operations in MATLAB. Such errors could include potential for finer iteration bounds in the Kepler's problem or errors due to the limit of decimals MATLAB can store. Overall, using MATLAB with the describe approach is still very effective considering MATLAB's strong compatibility with matrix and vector operations.

IV. Conclusion

Through analyzing the simulated scenario, a better understanding of the importance of the methodology and application of orbital mechanics was obtained. Each previous key found from just a few given details could be used to calculate the next bit of information until one arrived at a fairly accurate predicted location with detailed information about the specific orbit the satellite was in. Applications like this are done daily by groups such as NASA to track spacecrafts, satellites, and debris which are pivotal in conducting major space missions using similar given information. After designing a way to compute the satellites orbital elements as well as its initial and final position and velocity vectors using just the range, range rate, date, time of day, time of flight, and latitude and longitude of the observation station, there is a clear appreciation of how accurately and quickly such knowledge can be obtained using fundamental ideas brought about centuries ago. The use of sophisticated computer programs can build on those ideas to very accurately provide values required in analyzing orbital bodies.

V. Appendix

The DetOrbit.m function requires ten inputs and results in twelve outputs. All inputs can be obtained from the given simulation scenario where time is split into hour, minute, and second. The function is written on the same page as the script to provide reduced call time. To run DetOrbit.m, first open the VehiclePerformanceProject.m file attached with report submission. Next input the corresponding elements in the prompted variable definitions. Annotations have been provided to clarify units used and time zone necessary for computation. The order of inputs as well as outputs are provided in the function script so note the order of your inputs and outputs when calling the DetOrbit.m function to ensure accuracy of output

variables. Once the script is properly filled out, all that is left is to run the function in the command window in which all outputs can be viewed directly from the command window.